

GeoCAR Suite

Código-Fonte Python Completo e Comentado

Autor: Miguel A. L. Ferreira | Versão: 1.0 | Ambiente: QGIS LTR + Python 3.9+

Linkedin: www.linkedin.com/in/miguel-ferreira-43480b276

GitHub: https://github.com/geoferreira-source/GeoCAR_Suit.git

E-mail: geo.miguelferreira@gmail.com

1. Módulo de Autenticação — auth_manager.py

Responsável pela autenticação no portal SEMAS-PA via Selenium com captura de token Bearer JWT e cookies Cloudflare usando Chrome DevTools Protocol (CDP).

```
"""
auth_manager.py
Gerenciamento de autenticação no portal SEMAS-PA.
Utiliza Selenium + CDP para capturar o token Bearer JWT e o cookie Cloudflare.
"""

import time
import json
import logging
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

logger = logging.getLogger(__name__)

SEMAS_URL = "https://monitoramento.semas.pa.gov.br/simlam/"

class AuthManager:
    """Gerencia autenticação e captura de tokens no portal SEMAS-PA."""

    def __init__(self):
        self.bearer_token = None
        self.cf_cookie = None
        self.driver = None

    def _criar_driver(self):
        """Cria instância do Chrome com CDP habilitado."""
        opts = Options()
```

```
        opts.add_argument("--start-minimized")          # Não headless: necessário
para Cloudflare
        opts.add_argument("--disable-blink-features=AutomationControlled")
        opts.add_argument("--no-sandbox")
        opts.add_argument("--disable-dev-shm-usage")
        opts.add_experimental_option("excludeSwitches", ["enable-automation"])
        opts.add_experimental_option("useAutomationExtension", False)
        # Habilitar captura de rede via CDP
        opts.set_capability("goog:loggingPrefs", {"performance": "ALL"})
        svc = Service() # ChromeDriver no PATH
        self.driver = webdriver.Chrome(service=svc, options=opts)
        return self.driver

def autenticar(self, usuario: str, senha: str) -> bool:
    """
    Realiza login e captura tokens necessários.
    Retorna True se autenticação bem-sucedida.
    """
    try:
        driver = self._criar_driver()
        driver.get(SEMAS_URL)
        wait = WebDriverWait(driver, 30)

        # Aguardar campo de login aparecer (portal Angular)
        campo_usuario = wait.until(
            EC.presence_of_element_located((By.NAME, "username")))
        campo_usuario.clear()
        campo_usuario.send_keys(usuario)

        campo_senha = driver.find_element(By.NAME, "password")
        campo_senha.clear()
        campo_senha.send_keys(senha)

        btn_login = driver.find_element(By.CSS_SELECTOR,
"button[type='submit']")
        btn_login.click()

        # Aguardar redirecionamento pós-login
        time.sleep(4)

        # Estratégia 1: capturar token dos logs de performance (CDP)
        token = self._capturar_token_cdp(driver)

        # Estratégia 2: capturar token do localStorage
        if not token:
            token = self._capturar_token_localstorage(driver)

        # Estratégia 3: capturar token das cookies
        if not token:
```

```
        token = self._capturar_token_cookie(driver)

        # Estratégia 4: capturar token via requisição de teste
        if not token:
            token = self._capturar_token_request(driver)

        if not token:
            logger.error("Falha: token não encontrado em nenhuma
estratégia.")
            return False

        self.bearer_token = token
        self.cf_cookie = self._capturar_cf_cookie(driver)
        logger.info("Autenticação bem-sucedida.")
        return True

    except Exception as e:
        logger.error(f"Erro na autenticação: {e}")
        return False
    finally:
        if self.driver:
            self.driver.quit()

def _capturar_token_cdp(self, driver) -> str:
    """Extraí Bearer token dos logs de rede via CDP."""
    logs = driver.get_log("performance")
    for entry in logs:
        msg = json.loads(entry["message"])["message"]
        if msg.get("method") == "Network.requestWillBeSent":
            headers = msg.get("params", {}).get("request",
{}).get("headers", {})
            auth = headers.get("authorization", "")
            if auth.startswith("Bearer "):
                return auth.replace("Bearer ", "")
    return None

def _capturar_token_localstorage(self, driver) -> str:
    """Tenta recuperar token do localStorage do Angular."""
    try:
        token = driver.execute_script(
            "return localStorage.getItem('access_token') ||
localStorage.getItem('token');"
        )
        return token
    except Exception:
        return None

def _capturar_cf_cookie(self, driver) -> str:
    """Captura o cookie __cf_bm do Cloudflare."""
    for cookie in driver.get_cookies():
```

```
        if cookie["name"] == "__cf_bm":
            return cookie["value"]
    return None

def get_headers(self) -> dict:
    """Retorna headers completos para requisições à API SEMAS-PA."""
    return {
        "authorization": f"Bearer {self.bearer_token}",
        "x-explorer-account-token": "atos-autorizativos",
        "cookie": f"__cf_bm={self.cf_cookie}",
        "Content-Type": "application/json",
        "Accept": "application/json",
    }
```

2. Módulo de API — api_client.py

Gerencia as três chamadas principais à API SEMAS-PA: busca do CAR, download de geometrias GeoJSON e download do demonstrativo PDF.

```
"""
api_client.py
Cliente para os endpoints da API SEMAS-PA.
Endpoints: buscarCAR, consultaBuscarGeojson, baixarDemonstrativo
"""

import requests
import logging
from typing import Optional, Tuple

logger = logging.getLogger(__name__)

BASE_URL = "https://monitoramento.semas.pa.gov.br/simlam/api"

class APIClient:
    """Encapsula as chamadas à API REST do SEMAS-PA."""

    def __init__(self, headers: dict):
        self.headers = headers
        self.session = requests.Session()
        self.session.headers.update(headers)

    def buscar_car(self, numero_car: str) -> Optional[dict]:
        """
        Endpoint 1: buscarCAR
        Retorna o _id interno MongoDB do imóvel a partir do número CAR.
        """
        url = f"{BASE_URL}/car/buscarCAR"
```

```
payload = {"numeroCar": numero_car}
try:
    resp = self.session.post(url, json=payload, timeout=30)
    resp.raise_for_status()
    data = resp.json()
    if data and len(data) > 0:
        return data[0] # Retorna primeiro resultado
    logger.warning(f"CAR não encontrado: {numero_car}")
    return None
except requests.HTTPError as e:
    logger.error(f"HTTP {e.response.status_code} em buscarCAR:
{numero_car}")
    return None

def buscar_geojson(self, car_id: str) -> Optional[dict]:
    """
    Endpoint 2: consultaBuscarGeojsonDaAreaDoImovel
    Retorna geometrias do imóvel em formato GeoJSON.
    """
    url = f"{BASE_URL}/car/consultaBuscarGeojsonDaAreaDoImovel"
    payload = {"id": car_id}
    try:
        resp = self.session.post(url, json=payload, timeout=60)
        resp.raise_for_status()
        return resp.json()
    except requests.HTTPError as e:
        logger.error(f"HTTP {e.response.status_code} em buscarGeojson:
{car_id}")
        return None

def baixar_demonstrativo(self, car_data: dict) -> Optional[bytes]:
    """
    Endpoint 3: baixarVisualizarDemonstrativoDoCar
    Download do PDF demonstrativo – requer dois ObjectIds MongoDB.
    CRÍTICO: usar fileData._id (id1) + root-level id (id2).
    """
    try:
        id1 = car_data["fileData"]["_id"] # ObjectId do arquivo
        id2 = car_data["id"] # ObjectId raiz
        url = (
            f"{BASE_URL}/_classId/000000c819062a2fb5741d80"
            f"/_download/{id1}/{id2}"
        )
        resp = self.session.get(url, timeout=120)
        resp.raise_for_status()
        return resp.content
    except (KeyError, requests.HTTPError) as e:
        logger.error(f"Erro ao baixar demonstrativo: {e}")
        return None

def extrair_municipio(self, geojson_data: dict, numero_car: str) -> str:
```

```
"""Extraí nome do município a partir do GeoJSON ou do código IBGE."""
try:
    # Prioridade: campo direto no GeoJSON
    props = geojson_data.get("features", [{}])[0].get("properties", {})
    municipio = props.get("nom_municipio") or props.get("municipio")
    if municipio:
        return municipio

    # Fallback: dicionário IBGE pelo código no número do CAR
    codigo = numero_car.split("-")[1][:7]
    return MUNICIPIOS_PA.get(codigo, "Município não identificado")
except Exception:
    return "Erro ao extrair município"
```

3. Módulo de Análise Espacial — spatial_analysis.py

Executa as sobreposições geoespaciais do imóvel CAR contra as 8 camadas de referência, com suporte a análise PRODES por ano.

```
"""
spatial_analysis.py
Análises de sobreposição espacial usando GeoPandas + PyQGIS.
Calcula interseções com 8 camadas de referência.
"""

import json
import logging
from pathlib import Path
import geopandas as gpd
from shapely.geometry import shape

logger = logging.getLogger(__name__)

class SpatialAnalyzer:
    """Gerencia sobreposições espaciais do imóvel CAR."""

    def __init__(self, config: dict, plugin_dir: str):
        self.config = config
        self.plugin_dir = Path(plugin_dir)
        self._cache = {} # Cache de camadas para evitar recarregamento

    def _carregar_camada(self, nome: str) -> gpd.GeoDataFrame:
        """Carrega camada GeoPackage/SHP com cache em memória."""
        if nome in self._cache:
            return self._cache[nome]
        cfg = self.config["layers"][nome]
        caminho = self.plugin_dir / cfg["path"]
        gdf = gpd.read_file(str(caminho))
        # Garantir CRS uniforme: SIRGAS 2000 geográfico
```

```
        if gdf.crs is None or gdf.crs.to_epsg() != 4674:
            gdf = gdf.to_crs(epsg=4674)
        self._cache[nome] = gdf
        return gdf

def geojson_para_gdf(self, geojson_data: dict) -> gpd.GeoDataFrame:
    """Converte resposta GeoJSON da API em GeoDataFrame."""
    gdf = gpd.GeoDataFrame.from_features(geojson_data["features"])
    gdf = gdf.set_crs(epsg=4674)
    return gdf

def calcular_area_ha(self, gdf: gpd.GeoDataFrame) -> float:
    """Calcula área em hectares (reprojetando para CRS métrico)."""
    gdf_m = gdf.to_crs(epsg=31982) # SIRGAS 2000 / UTM 22S
    return float(gdf_m.geometry.area.sum() / 10_000)

def sobreposicao(self, imovel_gdf: gpd.GeoDataFrame,
                 nome_camada: str) -> dict:
    """
    Calcula sobreposição entre o imóvel e uma camada de referência.
    Retorna dict com área sobreposta em hectares e percentual.
    """
    try:
        camada = self._carregar_camada(nome_camada)
        intersecao = gpd.overlay(imovel_gdf, camada, how="intersection")
        if intersecao.empty:
            return {"area_ha": 0.0, "percentual": 0.0, "tem_sobreposicao":
False}

        area_sob = self.calcular_area_ha(intersecao)
        area_total = self.calcular_area_ha(imovel_gdf)
        pct = (area_sob / area_total * 100) if area_total > 0 else 0.0
        return {
            "area_ha": round(area_sob, 4),
            "percentual": round(pct, 2),
            "tem_sobreposicao": True
        }
    except Exception as e:
        logger.error(f"Erro sobreposição {nome_camada}: {e}")
        return {"area_ha": 0.0, "percentual": 0.0, "tem_sobreposicao":
False, "erro": str(e)}

def sobreposicao_prodes(self, imovel_gdf: gpd.GeoDataFrame) -> dict:
    """Sobreposição PRODES com breakdown por ano de desmatamento."""
    try:
        prodes = self._carregar_camada("PRODES")
        campo_ano = self.config["layers"]["PRODES"].get("year_field", "ano")
        intersecao = gpd.overlay(imovel_gdf, prodes, how="intersection")
        if intersecao.empty:
            return {"total_ha": 0.0, "por_ano": {}}
        resultado = {"total_ha": 0.0, "por_ano": {}}
        for ano, grupo in intersecao.groupby(campo_ano):
```

```
        area = self.calcular_area_ha(grupo)
        resultado["por_ano"][int(ano)] = round(area, 4)
        resultado["total_ha"] += area
        resultado["total_ha"] = round(resultado["total_ha"], 4)
        return resultado
    except Exception as e:
        logger.error(f"Erro PRODES: {e}")
        return {"total_ha": 0.0, "por_ano": {}}

def analisar_imovel(self, geojson_data: dict) -> dict:
    """
    Executa análise completa do imóvel contra todas as 8 camadas.
    Retorna dicionário consolidado com todos os resultados.
    """
    imovel_gdf = self.geojson_para_gdf(geojson_data)
    area_total = self.calcular_area_ha(imovel_gdf)
    camadas = [
        "UC_Federal", "UC_Estadual", "Terra_Indigena",
        "Assentamento_Federal", "Quilombo_Federal",
        "Embargo_IBAMA", "SIGEF"
    ]
    resultado = {"area_total_ha": round(area_total, 4)}
    for camada in camadas:
        resultado[camada] = self.sobreposicao(imovel_gdf, camada)
    resultado["PRODES"] = self.sobreposicao_prodes(imovel_gdf)
    return resultado
```

4. Módulo de Relatórios — report_generator.py

Gera o relatório Excel com código de cores, com abas separadas para resultados e o dashboard com 4 gráficos matplotlib.

```
"""
report_generator.py
Geração de relatório Excel com código de cores e dashboard de gráficos.
"""

import logging
from pathlib import Path
from datetime import datetime
import openpyxl
from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
from openpyxl.chart import BarChart, PieChart, Reference
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

logger = logging.getLogger(__name__)

COR_VERDE = "27AE60"
```



```
COR_AMARELO = "F39C12"
COR_VERMELHO = "E74C3C"
COR_CABECALHO = "1A5276"

class ReportGenerator:
    """Gera relatórios Excel e dashboards matplotlib."""

    def __init__(self, output_dir: str):
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def _cor_sobreposicao(self, area_ha: float) -> str:
        """Retorna cor baseada na área de sobreposição."""
        if area_ha == 0: return COR_VERDE
        if area_ha < 1.0: return COR_AMARELO
        return COR_VERMELHO

    def gerar_excel(self, resultados: list, nome_arquivo: str = None) -> str:
        """
        Gera planilha Excel com resultados de todos os CARs processados.
        resultados: lista de dicts com dados de cada CAR.
        """
        if not nome_arquivo:
            ts = datetime.now().strftime("%Y%m%d_%H%M%S")
            nome_arquivo = f"GeoCAR_Relatorio_{ts}.xlsx"
        caminho = self.output_dir / nome_arquivo
        wb = openpyxl.Workbook()
        ws = wb.active
        ws.title = "Resultados"

        # Cabeçalho
        colunas = [
            "Nº", "CAR", "Município", "Área Total (ha)",
            "UC Federal (ha)", "UC Estadual (ha)", "Terra Indígena (ha)",
            "PRODES Total (ha)", "Assentamento (ha)",
            "Quilombo (ha)", "Embargo IBAMA (ha)", "SIGEF (ha)", "Status"
        ]
        for col, titulo in enumerate(colunas, 1):
            cell = ws.cell(row=1, column=col, value=titulo)
            cell.font = Font(bold=True, color="FFFFFF", name="Arial",
size=10)
            cell.fill = PatternFill("solid", fgColor=COR_CABECALHO)
            cell.alignment = Alignment(horizontal="center", vertical="center",
wrap_text=True)
            ws.row_dimensions[1].height = 40

        # Dados
        for i, r in enumerate(resultados, 2):
            dados = [
```

```

        i - 1, r.get("numero_car", ""),
        r.get("municipio", ""),
        r.get("area_total_ha", 0),
        r.get("UC_Federal", {}).get("area_ha", 0),
        r.get("UC_Estadual", {}).get("area_ha", 0),
        r.get("Terra_Indigena", {}).get("area_ha", 0),
        r.get("PRODES", {}).get("total_ha", 0),
        r.get("Assentamento_Federal", {}).get("area_ha", 0),
        r.get("Quilombo_Federal", {}).get("area_ha", 0),
        r.get("Embargo_IBAMA", {}).get("area_ha", 0),
        r.get("SIGEF", {}).get("area_ha", 0),
        r.get("status", "OK"),
    ]
    for col, val in enumerate(dados, 1):
        cell = ws.cell(row=i, column=col, value=val)
        cell.font = Font(name="Arial", size=9)
        cell.alignment = Alignment(horizontal="center")
        # Colorir células de sobreposição
        if col in range(5, 13) and isinstance(val, float):
            cor = self._cor_sobreposicao(val)
            cell.fill = PatternFill("solid", fgColor=cor)
            cell.font = Font(name="Arial", size=9,
                             color="FFFFFF" if val > 0 else "000000")

    # Ajustar larguras
    ws.column_dimensions["A"].width = 5
    ws.column_dimensions["B"].width = 32
    ws.column_dimensions["C"].width = 22
    for col in "DEFGHIJKLM":
        ws.column_dimensions[col].width = 16
    wb.save(str(caminho))
    logger.info(f"Excel gerado: {caminho}")
    return str(caminho)

def gerar_dashboard(self, resultados: list) -> str:
    """Gera imagem PNG com 4 gráficos de análise."""
    fig, axs = plt.subplots(2, 2, figsize=(16, 12))
    fig.suptitle("GeoCAR Suite – Dashboard de Análise", fontsize=16,
fontweight="bold")

    cars    = [r.get("numero_car", f"CAR {i+1}") for i, r in
enumerate(resultados)]
    areas   = [r.get("area_total_ha", 0) for r in resultados]
    prodes  = [r.get("PRODES", {}).get("total_ha", 0) for r in resultados]
    uc_fed  = [r.get("UC_Federal", {}).get("area_ha", 0) for r in
resultados]
    ti      = [r.get("Terra_Indigena", {}).get("area_ha", 0) for r in
resultados]

    # Gráfico 1: Área total por CAR
    axs[0,0].bar(range(len(cars)), areas, color="#1A5276")
    axs[0,0].set_title("Área Total por Imóvel (ha)")

```

```
    axs[0,0].set_xticks(range(len(cars)))
    axs[0,0].set_xticklabels([c[-8:] for c in cars], rotation=45,
ha="right", fontsize=7)

# Gráfico 2: Sobreposição PRODES
cores = ["#E74C3C" if p > 0 else "#27AE60" for p in prodes]
axs[0,1].bar(range(len(cars)), prodes, color=cores)
axs[0,1].set_title("Sobreposição PRODES (ha)")
axs[0,1].set_xticks(range(len(cars)))
axs[0,1].set_xticklabels([c[-8:] for c in cars], rotation=45,
ha="right", fontsize=7)

# Gráfico 3: UC Federal e Terra Indígena
x = range(len(cars))
axs[1,0].bar([i - 0.2 for i in x], uc_fed, width=0.4, label="UC
Federal", color="#2980B9")
axs[1,0].bar([i + 0.2 for i in x], ti, width=0.4, label="Terra
Indígena", color="#8E44AD")
axs[1,0].set_title("UC Federal vs Terra Indígena (ha)")
axs[1,0].legend()
axs[1,0].set_xticks(list(x))
axs[1,0].set_xticklabels([c[-8:] for c in cars], rotation=45,
ha="right", fontsize=7)

# Gráfico 4: Pizza – distribuição de sobreposições
categorias = ["UC Federal", "UC Estadual", "Terra Indígena",
              "PRODES", "Assentamento", "Quilombo", "IBAMA", "SIGEF"]
totais = [
    sum(r.get("UC_Federal",{}).get("area_ha",0) for r in resultados),
    sum(r.get("UC_Estadual",{}).get("area_ha",0) for r in resultados),
    sum(r.get("Terra_Indigena",{}).get("area_ha",0) for r in
resultados),
    sum(r.get("PRODES",{}).get("total_ha",0) for r in resultados),
    sum(r.get("Assentamento_Federal",{}).get("area_ha",0) for r in
resultados),
    sum(r.get("Quilombo_Federal",{}).get("area_ha",0) for r in
resultados),
    sum(r.get("Embargo_IBAMA",{}).get("area_ha",0) for r in resultados),
    sum(r.get("SIGEF",{}).get("area_ha",0) for r in resultados),
]
if sum(totais) > 0:
    axs[1,1].pie(totais, labels=categorias, autopct="%1.1f%%",
                  colors=["#1A5276", "#2980B9", "#E74C3C", "#E67E22",
                           "#27AE60", "#8E44AD", "#C0392B", "#F39C12"])
axs[1,1].set_title("Distribuição das Sobreposições")

plt.tight_layout()
ts = datetime.now().strftime("%Y%m%d_%H%M%S")
caminho = str(self.output_dir / f"dashboard_{ts}.png")
plt.savefig(caminho, dpi=150, bbox_inches="tight")
plt.close()
logger.info(f"Dashboard gerado: {caminho}")
```

```
return caminho
```

5. Classe Principal do Plugin — geocar_suite.py

Ponto de entrada do plugin QGIS. Integra todos os módulos e gerencia o processamento em lote com QThread para não bloquear a interface.

```
"""
geocar_suite.py
Classe principal do plugin GeoCAR Suite para QGIS.
CRÍTICO: Todas as chamadas à API QGIS devem ocorrer na thread principal.
"""

from qgis.PyQt.QtCore import QThread, pyqtSignal, QObject
from qgis.PyQt.QtWidgets import QAction, QMessageBox
from qgis.core import (
    QgsProject, QgsVectorLayer, QgsLayerTreeGroup,
    QgsCoordinateReferenceSystem
)
from qgis.utils import iface
import logging
import json
from pathlib import Path

from .auth_manager import AuthManager
from .api_client import APIClient
from .spatial_analysis import SpatialAnalyzer
from .report_generator import ReportGenerator

logger = logging.getLogger(__name__)

class WorkerGeoCAR(QObject):
    """Worker executado em QThread para processamento em background."""
    progresso = pyqtSignal(int, str) # (percentual, mensagem)
    concluido = pyqtSignal(list) # lista de resultados
    erro = pyqtSignal(str) # mensagem de erro

    def __init__(self, numeros_car, auth_manager, config, plugin_dir,
output_dir):
        super().__init__()
        self.numeros_car = numeros_car
        self.auth_manager = auth_manager
        self.config = config
        self.plugin_dir = plugin_dir
        self.output_dir = output_dir

    def run(self):
        """Executa processamento em lote. NÃO chamar API QGIS aqui."""
```

```
api = APIClient(self.auth_manager.get_headers())
analyzer = SpatialAnalyzer(self.config, self.plugin_dir)
resultados = []
total = len(self.numeros_car)

for idx, numero_car in enumerate(self.numeros_car):
    try:
        pct = int((idx / total) * 100)
        self.progesso.emit(pct, f"Processando {numero_car}...")

        # 1. Buscar _id interno
        car_data = api.buscar_car(numero_car)
        if not car_data:
            resultados.append({"numero_car": numero_car, "status": "Não
encontrado"})
            continue

        # 2. Buscar geometrias GeoJSON
        geojson = api.buscar_geojson(car_data["_id"])
        if not geojson:
            resultados.append({"numero_car": numero_car, "status": "Sem
geometria"})
            continue

        # 3. Análise espacial
        resultado = analyzer.analisar_imovel(geojson)
        resultado["numero_car"] = numero_car
        resultado["municipio"] = api.extrair_municipio(geojson,
numero_car)

        resultado["geojson"] = geojson # Para carga no QGIS
        resultado["car_data"] = car_data # Para PDF
        resultado["status"] = "OK"
        resultados.append(resultado)

    except Exception as e:
        logger.error(f"Erro em {numero_car}: {e}")
        resultados.append({"numero_car": numero_car, "status": f"Erro:
{e}"})

self.progesso.emit(100, "Análise concluída.")
self.concluido.emit(resultados)

class GeoCarSuite:
    """Classe principal do plugin QGIS."""

    def __init__(self, iface):
        self.iface = iface
        self.plugin_dir = str(Path(__file__).parent)
        self.config = self._carregar_config()
```

```

        self.worker      = None
        self.thread      = None

    def _carregar_config(self) -> dict:
        cfg_path = Path(self.plugin_dir) / "config.json"
        with open(cfg_path, encoding="utf-8") as f:
            return json.load(f)

    def iniciar_processamento(self, numeros_car: list,
                               usuario: str, senha: str):
        """Ponto de entrada chamado pela interface gráfica."""
        # 1. Autenticar (pode ser em thread ou main – não usa QGIS API)
        auth = AuthManager()
        if not auth.autenticar(usuario, senha):
            QMessageBox.critical(None, "GeoCAR Suite", "Falha na autenticação
SEMÁS-PA.")
            return

        # 2. Criar worker e mover para QThread
        self.thread = QThread()
        self.worker = WorkerGeoCAR(
            numeros_car, auth, self.config,
            self.plugin_dir,
            self.config.get("output_dir", self.plugin_dir)
        )
        self.worker.moveToThread(self.thread)
        self.thread.started.connect(self.worker.run)
        self.worker.concluido.connect(self._on_fim)      # ← handoff para main
thread
        self.worker.progresso.connect(self._on_progresso)
        self.thread.start()

    def _on_progresso(self, pct: int, msg: str):
        """Atualiza barra de progresso na interface (main thread)."""
        self.iface.statusBarIface().showMessage(f"{pct}% – {msg}")

    def _on_fim(self, resultados: list):
        """
        CRÍTICO: Este slot executa na MAIN THREAD.
        Aqui é seguro chamar QgsVectorLayer e QgsProject.
        """
        self.thread.quit()
        self.thread.wait()

        # Carregar camadas no QGIS (APENAS aqui)
        for r in resultados:
            if r.get("status") == "OK" and r.get("geojson"):
                self._carregar_geojson_qgis(r["numero_car"], r["geojson"])

        # Gerar relatório Excel e dashboard

```

```
gen = ReportGenerator(self.config.get("output_dir", self.plugin_dir))
excel_path = gen.gerar_excel(resultados)
gen.gerar_dashboard(resultados)
QMessageBox.information(None, "GeoCAR Suite",
    f"Processamento concluído!\n{len(resultados)} CARs
    analisados.\nRelatório: {excel_path}")

def _carregar_geojson_qgis(self, numero_car: str, geojson: dict):
    """Carrega GeoJSON como camada vetorial no QGIS."""
    import json, tempfile, os
    tmp = tempfile.NamedTemporaryFile(
        mode="w", suffix=".geojson", delete=False, encoding="utf-8")
    json.dump(geojson, tmp)
    tmp.close()
    layer = QgsVectorLayer(tmp.name, numero_car, "ogr")
    if layer.isValid():
        QgsProject.instance().addMapLayer(layer)
    else:
        logger.warning(f"Camada inválida: {numero_car}")
    os.unlink(tmp.name)
```